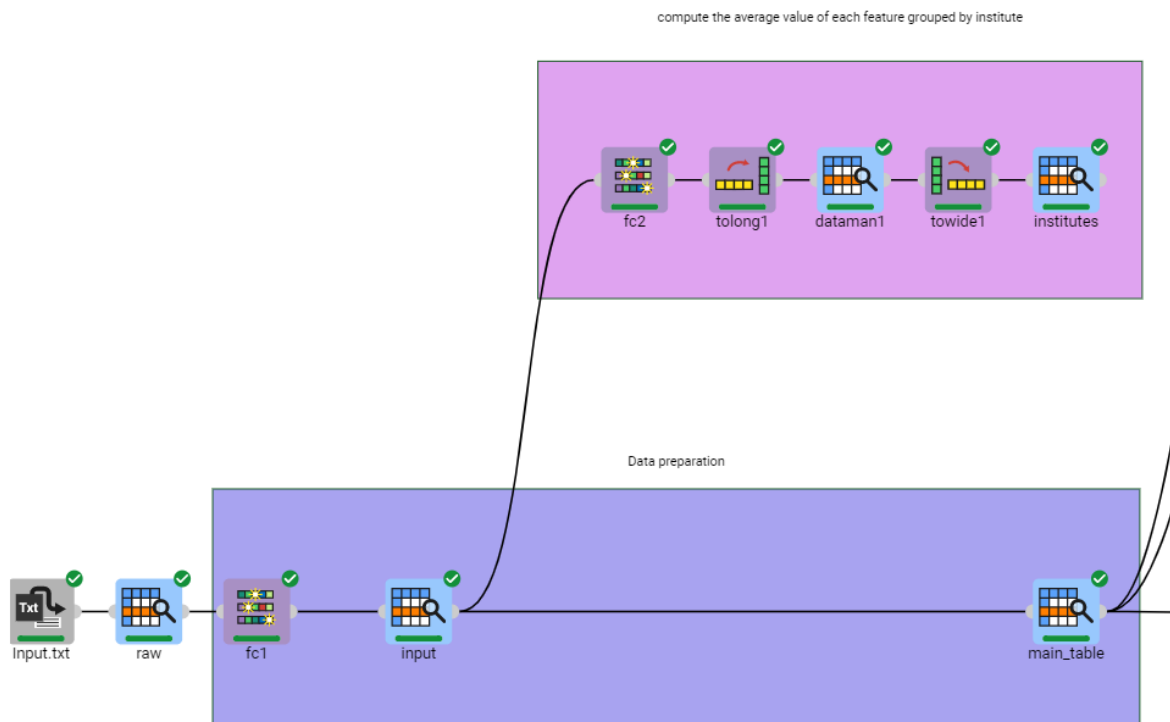


The image displays a KNIME workflow titled "Generate User-Defining Clusters". The workflow is organized into several main sections, each represented by a different colored background:

- Data preparation (Purple):** Starts with "Input DT" and "Date" nodes, followed by "SQL" and "Output" nodes. A "Data preparation" label is present.
- Results (Blue):** Contains a table with columns "cluster", "cluster2", and "cluster3".
- Select the best number of clusters (Green):** Contains a table with columns "cluster", "cluster2", and "cluster3".
- Generate User-Defining Clusters (Orange):** Contains a table with columns "cluster", "cluster2", and "cluster3".
- Cluster info (Red):** Contains a table with columns "cluster", "cluster2", and "cluster3".

The workflow includes various nodes such as "SQL", "Output", "Table", "Cluster", "Cluster2", and "Cluster3". The workflow is divided into several colored blocks: purple for data preparation, blue for results, green for cluster selection, orange for cluster generation, and red for cluster info.

Data Preparation and Analysis per Institute



The first part of the flow is subdivided in two parts:

- The lower one works to import and prepare the data for the clustering phases.
- The upper one is just about obtaining the average values per each attribute for each one of the institutes involved in the analysis.

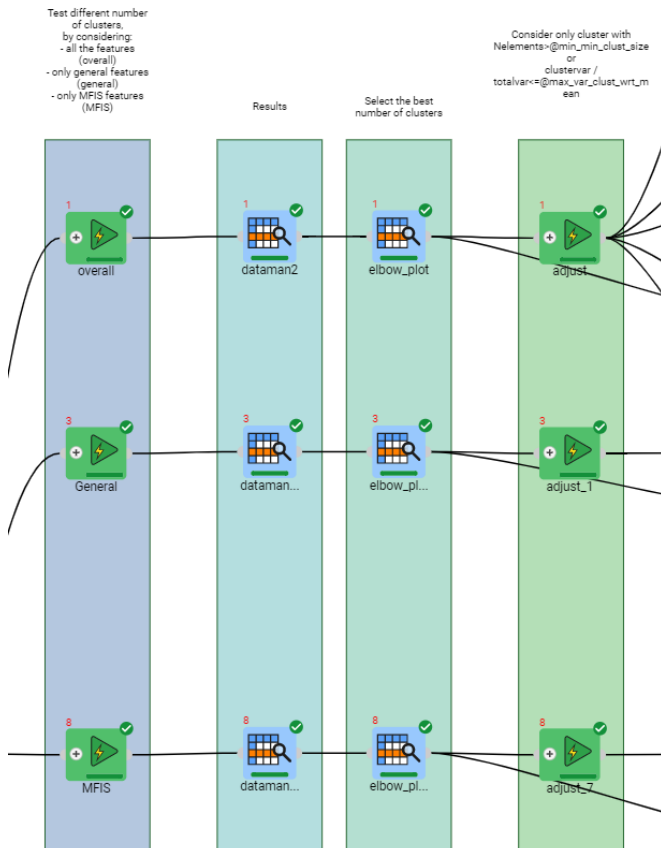
Data Preparation

In the Data Preparation part of the flow we first delete all the useless attributes with the *fc1* "Fill and Clean" task and later, in the *input* "Data Manager", we just ignore some of the attributes that must not be considered in the following procedures and with a long list of similar *ifelse()* functions we substitute the negative values, that are meaningless, with a *None* value. Finally, we apply a shift of -1 for a certain subgroup of attributes whenever we have a higher value respect to 3. We finally obtain our *main_table*, where we apply some changes of *type* for some attributes, we *ignore* or *delete* some other attributes and we fix for the *"Osteoporosis"* attribute the *None* value where we have 2 as a value.

Analysis per Institute

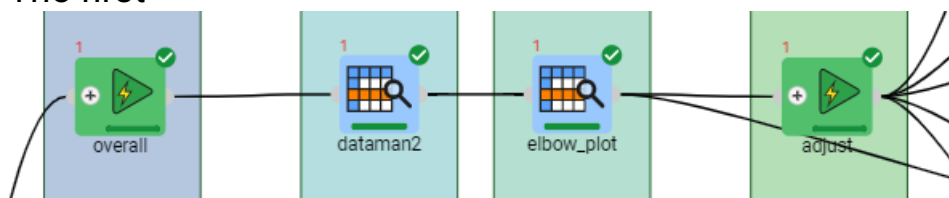
In the upper part of the flow (the pink box of the previous image), it is computed the average value of each of the attributes grouping by institute, except for three of the attributes that are removed through the *fc2* task.

Clustering



The second part of the flow is structured in three parallel parts, that apply the same pipeline to the data but considering different sets of input attributes.

The first

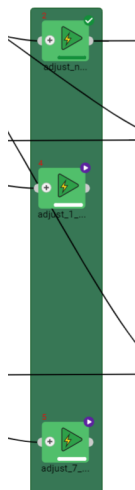


The *overall* module applies a set of separated clustering with the *K-Means* algorithm, iteratively considering different set of possible *K* and considering as input attributes for the clustering always all the input attributes.

The *K-Means* algorithm is a clustering algorithm that can achieve great results, but it has the big drawback that it needs a number *K* of clusters to retrieve that must be selected by the user apriori. This generate some issues, because typically the best number *K* of clusters to consider is unknown. So, the idea of finding the clusters considering different *K* is used to solve this issue: we try different *K* and after we compute a *Quality* value per each *K*, finding the best *K* respect to this *Quality*.

The *Quality* value per each *K* is computed inside the *elbow_plot* data manager. The *K Quality* is computed as the normalized difference between the intra-cluster distance variance with *K* clusters and with *K-1* clusters over the intra-cluster distance variance with *K-1* clusters, all over *K*.

The best *K* is the *K* that didn't give any single-element clusters and that have the max *Quality*.



The *adjust* modules, instead, consider only the clusters obtained through the best K and tries to optimize the clusters generated removing the clusters that don't satisfy the conditions respect to two thresholds. The first threshold is on the number of elements in a cluster and the second is on the inter-cluster variance.

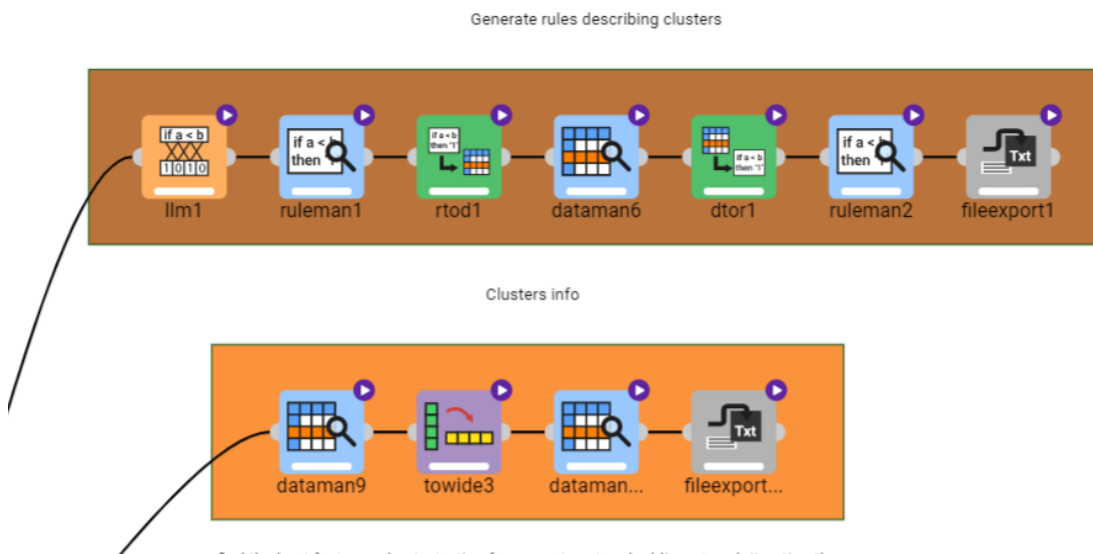
There is also a variant for the *adjust* modules, that is in the dark green box, that applies the same logics without considering the threshold on inter-cluster variance.

The second and the third

The second and the third parallel lines of flow, follow the exact same logic of the first line, but with the difference that they consider different subset of attributes in input for the clustering.

- The second line considers only the *general* features.
- The third line considers only the features recognized as part of *MFIS* features.

Rules and Info about clusters

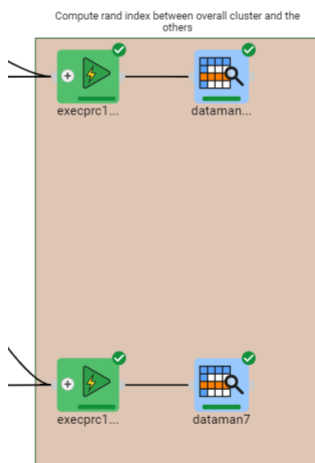


There is a beside part respect to the core part of the flow that is subdivided in two parts and that has task to retrieve informations about the adjusted clusters.

The first part (the brown one) uses a machine learning algorithm called *Logic Learning Machine* to extract a set of rules that identify in an intelligible way the main patterns of the data clustered together. The final extracted rules, after a brief cleaning phase, are readable in the *ruleman2* rule manager task.

The second part (the orange one) instead extracts a general overview about percentage of occurrences for the main features in each one of the clusters and the number of total patterns sex/ethnicity in each one of the clusters. The final table is readable in *dataman10* data manager.

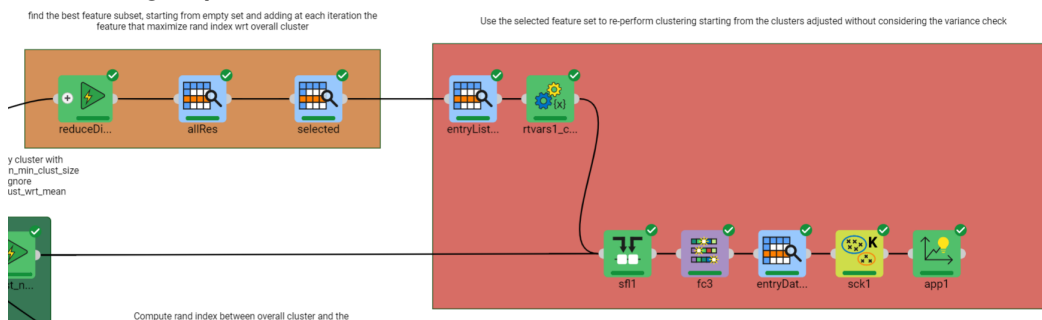
Rand Index



This part of the flow just computes the *Rand Index* between the clustering with all the input attributes and the clustering with only the general input attributes and between the clustering with all the input attributes and the clustering with only the MFIS attributes.

The *Rand Index* is an index of similarity between two clustering. If we have a *Rand Index* equal to 1, we are comparing two equal clustering. Otherwise, if we have a small value for the *Rand Index*, it tells us that our two clustering are very different.

Reducing Input Dimension



In the final part of the flow the main target is to find a new clustering with a new input list in an iterative way, that keeps the highest possible Rand Index with the clustering obtained considering all the features but reducing the number of features in input.

The core of this part of flow is inside the *reduceDimension* module, that with a “forward” or a “backward” approach, selected through the *@direction* process variable, tries to find a new list of attributes. The final list found with this approach that contains the input variables to use, is written inside the *entryLists_cluster* and automatically considered for a final clustering thanks to the *rtvars1_cpy* runtime variables task, that replaces, during the computation, the value of the *@reducedList* process variable.

The final clustering, that occurs in *sck1* k-means task, initializes the clustering with the clusters found previously considering all the input features and without the inter-variance threshold. This final clustering is applied in the *app1* task, and with a *take a look* on it we can see the final obtained clusters.